Café Exquisito

Memoria del proyecto

Memoria del Proyecto Final de Máster

Estudiante: [Escribe tu nombre completo] Programa: Máster en Desarrollo Full Stack Centro: Conquer Blocks Fecha de elaboración: 9 de noviembre de 2026 Aplicación publicada: https://proyecto-cafe-top-1.pages.dev/ API publicada: https://proyecto-cafe-top-1.onrender.com/

> El nombre del estudiante se deja para completar. La URL del repositorio, el enlace al vídeo y las credenciales de prueba se entregan en los campos o comentarios de la plataforma, no en este PDF. El documento separa las pruebas locales de las comprobaciones que se deben demostrar en la aplicación publicada.

Documento preparado para revisión y edición

Contenido

- Presentación del proyecto
- Resumen
- Contexto de la cafetería
- Problema que se quiere resolver
- Aportación del proyecto
- Objetivos y alcance
- Personas usuarias
- Casos de uso
- Recorrido y requisitos principales
- Diseño de la experiencia
- Catálogo y productos
- Personalización de bebidas y alimentos
- Carrito y pedidos
- Reservas, eventos y promociones
- Cuentas y permisos
- Panel de gestión
- Asistente de menú
- Tecnologías utilizadas
- Arquitectura e interfaz
- Modelo de datos
- API y reglas del servidor
- Despliegue de la aplicación
- Pruebas realizadas
- Seguridad y privacidad
- Accesibilidad y adaptación a pantallas
- Limitaciones actuales
- Decisiones y aprendizajes
- Conclusiones y mejoras futuras
- Guion del vídeo de defensa
- Fuentes y declaración de uso de IA

1. Presentación del proyecto

Café Exquisito es una aplicación web para una cafetería. Reúne en un mismo lugar la carta, información sobre productos, promociones, eventos, reservas y herramientas de gestión. La propuesta busca que una persona pueda conocer la oferta y preparar un pedido desde su navegador, y que el equipo pueda mantener parte del contenido sin editar directamente cada pantalla.

El proyecto se desarrolló como una aplicación web con una interfaz en React y una API creada con Node.js y Express. MongoDB almacena los datos de la versión desplegada. La interfaz se publica en Cloudflare Pages y el servidor de la API en Render. Esta separación permite actualizar la página y el servidor de manera independiente.

El sitio publicado puede consultarse en `https://proyecto-cafe-top-1.pages.dev/`. La API pública utiliza el dominio `https://proyecto-cafe-top-1.onrender.com/`. En esta memoria se explica la versión de Café Exquisito que se eligió para la publicación; no se presenta como un producto comercial terminado ni como un sistema que haya sido probado bajo carga real.

La carpeta del proyecto conserva el código de la interfaz y del servidor. La memoria describe la versión basada en React, Express y MongoDB, y no presenta como producción aquellas funciones que dependen de una configuración pendiente.

2. Resumen

Café Exquisito comenzó con una idea sencilla: presentar una carta de cafetería de forma atractiva y permitir que varias tareas habituales se consulten desde el mismo sitio. A partir de esa idea se construyeron una portada, un catálogo por categorías, un carrito, flujos de pedidos, formularios de reserva y secciones de promociones y eventos.

La aplicación tiene dos partes principales. La primera es la interfaz que usa el público, realizada con React y Vite. La segunda es una API en Express que organiza las operaciones y consulta MongoDB. El navegador no se conecta directamente a la base de datos: envía solicitudes al servidor, que devuelve respuestas para la página.

También se añadieron cuentas con distintos roles, opciones de personalización de productos, un área administrativa y funciones que dependen de servicios configurables, como imágenes alojadas externamente o un asistente conectado a un proveedor de IA. Algunas de esas capacidades requieren variables privadas y no deben describirse como activas si no se han probado en la publicación final.

El trabajo se validó mediante compilación del frontend, pruebas automatizadas disponibles y una comprobación manual de flujos en un entorno local. Estas comprobaciones son útiles, pero no equivalen a probar todas las funciones con cuentas reales y servicios de producción. En la presentación se debe distinguir con claridad lo que está desplegado, lo que se probó localmente y lo que continúa como mejora.

Palabras clave: cafetería, aplicación web, React, Express, MongoDB, pedidos, catálogo.

3. Contexto de la cafetería

Una cafetería suele comunicar su oferta por varios medios: una carta impresa, redes sociales, mensajes directos o atención en el local. Cuando la información está dispersa, una persona puede tener que preguntar por precios, ingredientes, disponibilidad o promociones antes de decidir qué pedir.

Café Exquisito plantea una experiencia digital para organizar esa información. La portada presenta el concepto del negocio y guía a la persona hacia el menú. La carta se divide en categorías, como bebidas, frappes, desayunos, lunch y repostería. También hay espacios para conocer promociones, eventos, reservas y otros aspectos del establecimiento.

La historia y los textos del sitio forman parte de la identidad visual del proyecto. Se busca transmitir un ambiente cálido, con una paleta oscura y tonos de café y caramelo. El propósito no es sustituir la atención de una persona barista, sino facilitar la consulta previa y ordenar algunos pasos del servicio.

El problema se plantea como una oportunidad de diseño académico. Esta memoria no atribuye opiniones a clientes ni presenta entrevistas o encuestas que no estén documentadas.

4. Problema que se quiere resolver

El problema de diseño abordado es la dificultad de consultar una oferta amplia y, después, comunicar con claridad qué desea la persona. Una carta digital por sí sola puede quedarse corta si no muestra categorías comprensibles, precios, disponibilidad u opciones de personalización. Del mismo modo, un botón para agregar un producto no resuelve el flujo si no se entiende qué se añadió o cuánto cuesta el pedido.

El proyecto organiza la información del menú y relaciona la selección con un carrito que muestra productos, cantidades y total. En el caso de ciertos productos, las opciones seleccionadas se conservan junto con el pedido para que no se pierdan al pasar de la interfaz al servidor.

También se considera el trabajo de administración. Si los productos cambian, resulta más práctico disponer de un panel para mantener algunos datos que modificar manualmente el código de cada tarjeta. La aplicación incluye herramientas administrativas, aunque cada permiso y operación debe probarse con las cuentas adecuadas antes de asegurar que cubre cualquier situación real de una cafetería.

El objetivo de esta sección no es afirmar que el sistema elimina errores o reduce tiempos en una cantidad determinada. Para hacer esas afirmaciones sería necesario medir tiempos, comparar un proceso anterior con uno posterior y describir las condiciones de la medición. En ausencia de esa evidencia, el beneficio se presenta como una posibilidad que debe evaluarse con usuarios.

Pregunta para la defensa: ¿qué parte del recorrido actual del cliente es más clara gracias a la página y qué parte todavía requiere atención humana?

5. Aportación del proyecto

La solución es un sitio adaptable a distintos tamaños de pantalla, con una portada que presenta el negocio y un menú de productos consultable por categorías. Desde las tarjetas se abre el flujo de personalización correspondiente. Después, la persona puede revisar el carrito y comprobar los artículos y el total antes de confirmar el pedido.

La propuesta incluye otras secciones que complementan la experiencia: promociones, actividades, reservas, información del equipo y contenido institucional. El panel de gestión reúne herramientas para mantener información que la aplicación permite editar. Las cuentas permiten distinguir a clientes de determinados roles internos.

La solución se separa en frontend y backend. El frontend muestra la información y recoge las selecciones. La API valida y procesa las operaciones del servidor y MongoDB conserva los registros. Esta separación hace posible cambiar la presentación sin conectar la interfaz directamente a la base de datos.

La aplicación también contempla estados alternativos: productos no disponibles, errores de conexión, imágenes que no cargan y recursos guardados por el navegador. El modo sin conexión es limitado; conservar algunos archivos o productos consultados no significa que se puedan confirmar pedidos sin internet o sincronizarlos después.

La aportación consiste en reunir en una sola experiencia la consulta del menú, la personalización según cada producto, la revisión del carrito y varios apartados de atención y gestión. También se cuidó que las elecciones del cliente sigan asociadas al artículo al pasar de la interfaz al servidor.

La interfaz y sus funciones se desarrollaron de forma incremental, atendiendo problemas observados durante las pruebas, como la claridad del carrito, las opciones de los productos y la navegación hacia el menú. En la defensa se debe mostrar únicamente el comportamiento que se encuentre en el despliegue que se presenta.

6. Objetivos y alcance

Objetivo general

Crear una aplicación web que presente la oferta de Café Exquisito y reúna flujos básicos de consulta, selección de productos, pedidos, reservas y administración de contenido.

Objetivos específicos

- Organizar los productos por categorías y mostrar su precio y descripción.
- Permitir que se seleccionen opciones relacionadas con el producto.
- Mantener visible el contenido del carrito y su total.
- Conectar la interfaz con una API y una base de datos.
- Diferenciar ciertas acciones según el rol de la cuenta.
- Hacer que la página pueda consultarse desde pantallas de escritorio y móviles.
- Probar la compilación, reglas principales y flujos relevantes antes de presentar el proyecto.

Alcance

La entrega se centra en la aplicación web y en la publicación de su interfaz y API. No incluye una operación comercial real, una pasarela de pago confirmada, un programa de producción de alimentos ni métricas verificadas de clientes reales. La disponibilidad de servicios opcionales depende de su configuración.

Las funciones que mejor resumen el alcance son: catálogo, personalización, carrito y pedido, reservas, promociones y eventos, y administración. La integración de pagos y la operación comercial real no forman parte de lo que esta memoria afirma como verificado.

7. Personas usuarias

La página contempla varios tipos de personas. El visitante consulta el contenido público sin iniciar sesión. El cliente registrado puede utilizar las funciones vinculadas a su cuenta, como el historial de pedidos, según el modo habilitado en el despliegue. El personal puede necesitar herramientas de atención o de gestión. Los encargados y administradores cuentan con permisos más amplios según el rol.

En el código del backend aparecen roles como `client`, `barista`, `atencion`, `encargado`, `subgerente`, `gerente`, `supervisor`, `codueno` y `admin`. Que estos nombres existan en el modelo no demuestra por sí solo que cada rol tenga exactamente los permisos esperados. La autorización se debe revisar en las rutas y probar con cuentas de prueba separadas.

Para una presentación segura conviene preparar cuentas sin datos personales reales. Las contraseñas no se deben incluir en el PDF, en capturas ni en el repositorio. Si se requieren credenciales de evaluación, deben compartirse por un canal privado autorizado y retirarse después de la defensa.

Perfil	Necesidad principal	Ejemplo de recorrido
Visitante	Conocer la propuesta y la carta	Abre la portada y consulta categorías
Cliente	Elegir y revisar productos	Personaliza artículos y revisa el carrito
Personal	Atender o consultar operaciones	Revisa la información que le permite su rol
Administración	Mantener contenido	Actualiza productos u otros datos habilitados

Evidencia pendiente: documenta qué acciones probaste con cada rol y qué respuesta obtuviste.

8. Casos de uso

Los casos de uso describen interacciones previstas en el producto. La tabla resume el flujo y el resultado esperado; no sustituye la demostración de que cada uno se comprobó en la versión publicada.

Persona	Caso de uso	Resultado esperado
Visitante	Abrir el sitio y recorrer el menú	Consulta categorías, productos y precios públicos
Cliente	Elegir un producto y ajustar sus opciones	El carrito conserva la configuración y muestra el total
Cliente	Enviar una solicitud de pedido o reserva	El servidor recibe la información y devuelve un estado
Personal autorizado	Consultar operaciones asignadas a su rol	Ve la información que sus permisos permiten
Administrador	Mantener productos y contenido del sitio	Los cambios se guardan mediante las funciones autorizadas

El sistema también contempla respuestas ante errores de red, productos no disponibles y sesiones vencidas. La autorización se valida en el servidor; ocultar un botón en la página no basta para restringir una operación.

Los casos de carga, disponibilidad prolongada, recuperación de respaldos y uso simultáneo por muchas personas requieren pruebas específicas y no se consideran demostrados por abrir la página.

9. Recorrido y requisitos principales

Los requisitos principales son presentar una portada, organizar productos por categorías, permitir una personalización pertinente, reflejar cantidades y total en el carrito, enviar operaciones a la API, ofrecer solicitudes de reserva y separar las tareas según la cuenta. La interfaz también debe presentar mensajes comprensibles si algo falla.

Un recorrido típico comienza en la portada. La persona puede explorar el contenido principal y bajar hasta el menú. Allí observa categorías y tarjetas de productos. Si pulsa la acción de agregar, se abre una ventana de personalización cuando el producto tiene opciones; el total se actualiza con las decisiones que afectan el precio.

Al confirmar la selección, el producto se incorpora al carrito. En la versión local probada, la incorporación muestra una pequeña animación hacia el icono del carrito y luego abre el panel para que la persona compruebe el resultado. Desde el carrito se pueden revisar las líneas, ajustar cantidades, vaciar el contenido y continuar hacia el menú si se encuentra vacío.

El siguiente paso de confirmación depende de la sesión, la conexión y el backend habilitado. Por ello, es importante ensayar el flujo con la cuenta de demostración y los servicios que se usarán durante la presentación. La memoria no debe sugerir que se procesó un pago si el pedido se paga en mostrador o si no se configuró la pasarela.

Las reservas siguen un recorrido aparte: la persona completa los datos solicitados y el servidor conserva la solicitud con un estado. El sitio también permite consultar promociones y eventos, aunque su disponibilidad depende de los registros y de la configuración del servicio.

Los requisitos de rendimiento, disponibilidad y restauración necesitan pruebas aparte; no se dan por cumplidos únicamente porque este recorrido funcione.

10. Diseño de la experiencia

La identidad visual utiliza una base oscura con tonos cálidos asociados al café. Los encabezados serif ayudan a dar personalidad a las secciones, mientras que los controles y textos de interfaz usan una tipografía sans serif para facilitar la lectura. Las tarjetas muestran primero la imagen y la categoría, y después el nombre, la descripción, el precio y la acción disponible.

La portada contiene varias secciones para que el sitio no sea solamente una lista de productos. La navegación superior permite cambiar de apartado, y hay accesos que llevan a partes concretas de la página principal. En pantallas más pequeñas, la navegación se reorganiza para no ocupar todo el ancho.

El carrito se presenta como un panel lateral sobre la página. Este patrón permite revisar el pedido sin perder por completo el contexto de la carta. En las tarjetas, el flujo de personalización aparece en una ventana superpuesta y mantiene a la vista el precio que se va a agregar.

Durante el desarrollo se hicieron ajustes para que el total fuera visible y para que la acción de volver al menú realmente desplazara la página al catálogo. La versión local actual también incluye una animación breve al añadir productos. Conviene comprobar el mismo comportamiento en el despliegue final antes de incluirlo como parte de la demostración grabada.

La paleta oscura con tonos cálidos, los títulos serif y los controles sencillos buscan que el sitio recuerde el ambiente de una cafetería sin dificultar la lectura. El menú y el carrito conservan acciones visibles para que la persona pueda seguir el recorrido sin perder sus elecciones.

11. Catálogo y productos

El catálogo organiza productos por categorías como bebidas, frappes, desayunos, lunch, repostería y café en grano. Los registros de producto incluyen nombre, precio, descripción, categoría, dirección de imagen, disponibilidad y, cuando aplica, opciones de personalización.

La página obtiene productos desde la API. Durante el desarrollo también existe un conjunto de datos compartidos para ciertos estados locales o de respaldo. La interfaz no debe confundir un dato de ejemplo con un registro confirmado en la base de producción; al probar, hay que observar si el catálogo carga desde el servicio publicado.

Las imágenes de productos se guardan en la carpeta pública del frontend o se sirven desde una ruta configurada. El sistema de sustitución corrige algunas rutas antiguas específicas cuando coinciden el nombre del producto y la imagen anterior. Además, se preparó un respaldo por categoría para evitar que una foto rota deje un espacio vacío.

En el catálogo real, la imagen debe corresponder al artículo anunciado. Si se utiliza una fotografía ilustrativa, es mejor indicarlo o incorporar créditos y sustituirla por una fotografía propia cuando sea posible. Las imágenes no deben elegirse únicamente por nombre de archivo sin revisar lo que representan.

Comprobación recomendada: abre productos de varias categorías, confirma que la foto carga y que el texto alternativo identifica el producto. Si la imagen no corresponde, anótalo como pendiente en lugar de presentarla como verificada.

12. Personalización de bebidas y alimentos

La personalización se ajusta al tipo de producto. En bebidas se pueden mostrar opciones como temperatura, leche, nivel de azúcar, jarabe o textura, cuando corresponden. Los frappes tienen opciones para el tipo de leche y el nivel de frappé. En alimentos y repostería puede aparecer la opción de calentar el producto. No se pregunta por leche para un refresco si esa decisión no forma parte de su preparación.

Las opciones visibles se construyen a partir de reglas compartidas para los catálogos y de la información del producto. La interfaz calcula el precio que se muestra, y el backend debe validar los datos y aplicar las reglas del servidor al recibir el pedido. Las opciones elegidas se guardan como parte de la línea del pedido, por lo que pueden mostrarse en el carrito, el historial y la comanda.

La propuesta intenta que la selección sea clara: las alternativas están agrupadas, la opción elegida se distingue y los recargos se muestran antes de confirmar. Los valores sugeridos no deben impedir que el cliente cambie su elección.

La corrección del precio entre interfaz y servidor es importante. El navegador puede mostrar una estimación, pero no debe ser la autoridad final de importes. Cualquier cambio en las tarifas debe reflejarse de forma coherente en catálogo, frontend, servidor e historial.

Caso para mostrar en el vídeo: selecciona un frappe, cambia el tipo de leche o la textura y explica cómo se refleja la selección en la línea del carrito. No incluyas una opción que no esté en el build publicado.

13. Carrito y pedidos

El carrito agrupa artículos y muestra el nombre, la cantidad, las opciones elegidas y el total. La persona puede aumentar o reducir cantidades, quitar una línea y vaciar el carrito. Una línea distinta se conserva separada cuando cambian sus opciones, para no mezclar, por ejemplo, dos bebidas preparadas de forma diferente.

La identificación de la línea incluye el producto y los valores de personalización relevantes, como tamaño, leche, extras, notas y opciones agrupadas. Al volver a agregar la misma combinación, se incrementa su cantidad dentro de un límite. Este comportamiento facilita revisar el pedido sin perder los detalles de preparación.

Cuando el cliente confirma un pedido, la interfaz prepara los datos y los envía al backend correspondiente. El servidor almacena el pedido asociado a la cuenta e incluye el tipo de servicio, los productos, el total y el estado. El modelo contempla estados como pendiente, en preparación, listo, entregado y cancelado.

El pedido no debe confundirse con un pago. El esquema tiene campos para registrar el estado de pago, pero la existencia de esos campos no prueba que una transacción real se haya completado. Si la demostración no utiliza una pasarela configurada, se debe decir expresamente que el pago se realiza fuera de la aplicación o que la integración no se incluye en el flujo mostrado.

El vídeo puede mostrar el carrito con datos de prueba. No se incluyen pagos ni transacciones reales como parte de las comprobaciones descritas.

14. Reservas, eventos y promociones

La aplicación incluye apartados para reservas, eventos y promociones. Una reserva recoge información de contacto, fecha, hora, número de personas y notas. En el modelo del servidor, el tamaño del grupo está limitado y la solicitud comienza con estado pendiente; puede pasar a confirmada o rechazada según la gestión que realice el equipo.

Los eventos permiten presentar actividades y orientar a la persona sobre fecha, hora, ubicación, capacidad y precio. Las promociones describen ofertas o combinaciones de productos. Estas secciones hacen que la portada comunique más que una carta y permiten destacar actividades de temporada.

La disponibilidad de reservas y eventos depende de cómo se mantenga el contenido y de las reglas del servidor. Una solicitud registrada no significa que la mesa haya quedado confirmada automáticamente. El sitio no debe anunciar disponibilidad confirmada sin una comprobación real del personal o del sistema correspondiente.

La memoria no da por comprobada una reserva en producción. Si se muestra en el vídeo, usa datos ficticios y confirma en el momento que la solicitud llega al sistema y qué estado recibe.

15. Cuentas y permisos

El registro y el inicio de sesión permiten asociar ciertas acciones a una cuenta. El modelo de usuario conserva nombre, correo, contraseña con hash, teléfono y rol. También contempla datos de fidelidad y otros atributos para cuentas internas.

El acceso a las rutas protegidas debe comprobarse en el servidor. Ocultar un botón en la interfaz no es suficiente para impedir una operación: una persona podría intentar llamar directamente a la API. Por ello, cada función administrativa necesita middleware de autenticación y autorización adecuado.

La contraseña se transforma con bcrypt antes de guardarse. Al iniciar sesión, el backend compara la contraseña introducida con su hash. La sesión de Express utiliza un token JWT. El cliente almacena datos de sesión localmente, de modo que se deben considerar los riesgos de exposición asociados al almacenamiento en el navegador y proteger la interfaz frente a contenido malicioso.

El modelo incluye perfiles públicos del equipo con opciones de consentimiento y visibilidad. Si se demuestra esta función, se debe comprobar que los datos personales no aparezcan públicamente por accidente y que la autorización para publicar un perfil esté confirmada.

Las acciones previstas por tipo de cuenta se resumen a continuación. La tabla describe el control esperado y no afirma que cada combinación de rol se haya ensayado en producción.

Perfil	Acceso previsto	Límite de acceso
Visitante	Secciones públicas	No administra registros
Cliente	Funciones asociadas a su cuenta	No consulta datos de otras cuentas ni administra contenido
Personal autorizado	Operaciones habilitadas para su rol	No realiza acciones reservadas a administración
Administrador	Funciones de gestión autorizadas	El acceso sigue sujeto a las reglas del servidor

Las credenciales de evaluación se comparten por el comentario privado o el canal indicado en la plataforma, nunca dentro de este PDF.

16. Panel de gestión

El panel administrativo concentra herramientas para operar el contenido y ciertas funciones internas de la aplicación. Entre las áreas presentes en el frontend se encuentran la gestión de productos, pedidos, usuarios, promociones, eventos, reservas, equipo y ajustes del sitio, de acuerdo con los módulos disponibles en el proyecto.

El panel resulta útil para mostrar cómo se separa la experiencia pública de las herramientas internas. Un cliente consulta la carta, mientras que una cuenta autorizada puede realizar acciones de gestión. La disponibilidad de cada sección depende del rol y de las rutas a las que se conecta.

La información sensible no debe mostrarse en capturas públicas. Para la demo se recomienda entrar con una cuenta de prueba, elegir una operación sencilla y comprobar que el cambio aparece en la pantalla pública o en el registro correspondiente. Después, se debe confirmar que un rol sin permisos no puede ejecutar la misma acción.

El panel no sustituye un procedimiento de respaldo ni un sistema de control de cambios. Antes de modificar registros importantes se debe conocer qué operación actualiza o elimina datos, y evitar probar comandos destructivos contra la base de producción. En el vídeo solo se debe mostrar una acción que se haya comprobado con una cuenta autorizada.

17. Asistente de menú

El proyecto contempla un asistente que ayuda a consultar el menú y responder preguntas relacionadas con la oferta. La integración con un proveedor externo, como Gemini, depende de que se configure una clave privada en el servidor. Esa clave no debe incorporarse al código del frontend ni subirse al repositorio.

Una recomendación automatizada debe basarse en los productos disponibles y en información real. Si la API no está configurada o no responde, la aplicación debe dar un mensaje claro y mantener una forma alternativa de consultar el menú. El asistente no reemplaza al personal ni debe considerarse una fuente suficiente para confirmar alergias, contaminación cruzada o necesidades médicas.

La disponibilidad de respuestas depende de la configuración del proveedor y del estado de la API. Una recomendación automatizada no garantiza ingredientes ni descarta alergias; esa información debe confirmarse con el personal.

El desarrollo incluye lógica para relacionar preguntas con elementos de menú y facilitar la navegación hacia secciones de la página. Esta función puede servir como ayuda para explorar, pero su calidad depende de los datos disponibles, la configuración del proveedor y la claridad de la pregunta.

El asistente se presenta como una función opcional y no es necesario incluirlo en la defensa si no se ha comprobado con la configuración activa.

18. Tecnologías utilizadas

La aplicación combina herramientas de frontend, servidor y almacenamiento. Las versiones mayores y librerías principales que aparecen en los manifiestos del proyecto son:

Parte	Tecnologías
Interfaz web	React 18, React DOM 18 y Vite 6
Estilos e iconos	Tailwind CSS 3 y Lucide React
Solicitudes desde la interfaz	Axios
Servidor	Node.js y Express 4
Datos	MongoDB y Mongoose 8
Acceso y protección de datos	JWT, bcrypt y validación de solicitudes
Pruebas	`node:test` para los módulos JavaScript
Publicación descrita	Cloudflare Pages, Render y MongoDB Atlas

Los archivos de dependencias del frontend y backend registran los paquetes del proyecto. Las variables privadas, como la conexión con MongoDB, se configuran en el servicio del servidor y no deben copiarse a la aplicación web.

Arquitectura general

La aplicación sigue una arquitectura cliente-servidor separada. El navegador descarga los archivos estáticos del frontend desde Cloudflare Pages. React construye la interfaz y realiza solicitudes HTTP al servicio de Express publicado en Render. Express aplica las reglas de negocio y consulta MongoDB.



La variable de entorno `VITE_API_URL` indica al frontend la dirección base de la API durante la compilación. CORS limita los orígenes permitidos mediante la configuración del backend. La API utiliza el puerto asignado por la plataforma en `PORT`.

El modo de desarrollo puede usar proxies locales. Esos proxies no deben confundirse con el enrutamiento de producción: en el entorno público, las direcciones de frontend y backend pertenecen a servicios distintos.

19. Arquitectura e interfaz

La aplicación usa una arquitectura cliente-servidor. El navegador recibe el frontend publicado y React presenta las páginas. Cuando necesita el menú o debe enviar una operación, la interfaz realiza una solicitud HTTPS a la API; Express procesa la petición y consulta MongoDB. El navegador no se conecta directamente a la base de datos.

El frontend se encuentra en la carpeta `frontend`. Está creado con React y Vite y organiza la aplicación en componentes, páginas, contextos y utilidades. Algunos componentes centrales son la barra de navegación, el catálogo, las tarjetas de producto, el carrito, el acceso a la cuenta y los mensajes de recuperación.

El contexto del carrito mantiene los artículos seleccionados, cantidades, suma total y apertura del panel. El contexto de autenticación comparte la sesión con las páginas y componentes que necesitan saber qué cuenta está activa. La separación evita tener que pasar cada dato manualmente por todas las capas de la interfaz.

Las páginas principales incluyen Inicio, pedidos, perfil, administración, eventos, promociones, reservas, equipo e información del negocio. El componente principal decide cuál se muestra y facilita la navegación interna. En la versión SPA, algunas secciones se presentan sin recargar todo el documento.

La interfaz gestiona datos de carga, errores y estados vacíos. Por ejemplo, si la API no responde, se conserva un mensaje de recuperación en vez de asumir que la consulta fue exitosa. Estas respuestas deben revisarse tanto en desktop como en móvil para que no queden ocultas ni fuera de pantalla.

Elementos para mostrar en el video: portada, navegación, filtro de categoría, tarjeta de producto y panel lateral del carrito.

20. Modelo de datos

MongoDB almacena documentos descritos por modelos de Mongoose. Las entidades principales son:

Modelo	Datos que representa	Uso en la aplicación
`User`	Cuenta, hash de contraseña, rol y datos asociados	Autenticación y permisos
`Product`	Nombre, categoría, precio, disponibilidad e información de personalización	Catálogo y validación de artículos
`Order`	Usuario, líneas, cantidades, opciones, total y estado	Registro y seguimiento del pedido
`Reservation`	Datos de contacto, fecha, hora, grupo y estado	Solicitud de mesa
`Promotion`	Información y vigencia de una promoción	Contenido público
`Event`	Actividad, fecha, lugar y capacidad	Agenda de eventos

La aplicación incluye además modelos para solicitudes de personal y ajustes del sitio. El servidor relaciona los registros cuando corresponde y valida la información antes de guardarla. El total que aparece en pantalla no convierte al navegador en autoridad para aceptar importes.

Los datos de demostración deben ser ficticios. Las exportaciones que contengan información de personas se deben mantener privadas y las operaciones de migración requieren una copia de seguridad.

21. API y reglas del servidor

El backend vive en la carpeta `backend` y su punto de entrada es `server.js`. Express registra rutas para autenticación, productos, pedidos, inteligencia artificial, eventos, promociones, reservas, personal y ajustes del sitio. La aplicación configura middleware de seguridad HTTP, límite de solicitudes para la API, CORS, lectura de JSON y manejo de errores.

El servidor inicia la conexión a MongoDB y expone una ruta de estado de salud. La ruta `/health` responde con un estado saludable cuando la conexión está lista y con un error de servicio cuando la base de datos no se encuentra conectada. Esta respuesta sirve como comprobación técnica básica; no prueba cada operación del sitio.

Las reglas de negocio deben permanecer en el servidor cuando afectan autorización, precios o escritura de datos. El cliente puede calcular una vista previa para que la persona entienda cuánto costaría una selección, pero el backend debe verificar el pedido recibido antes de guardarlo.

Las rutas no existentes y las excepciones internas pasan por middleware específico de errores. Al demostrar un flujo, es útil enseñar una solicitud correcta y, si el tiempo lo permite, cómo se informa de un dato inválido o de una falta de conexión. No se recomienda mostrar secretos, tokens ni contenido privado de la base.

22. Despliegue de la aplicación

La publicación separa la página web de la API. Cloudflare Pages sirve el frontend construido por Vite. Render ejecuta el backend Node.js. MongoDB Atlas proporciona la base de datos remota. La dirección compartida para la interfaz es `https://proyecto-cafe-top-1.pages.dev/`.

El frontend utiliza `VITE_API_URL` para localizar la API al compilar. El backend usa variables privadas como `MONGO_URI`, `JWT_SECRET` y `CORS_ORIGINS`. Las variables que empiezan por `VITE_` se incluyen en el código enviado al navegador, por lo que no deben guardar contraseñas ni claves privadas.

El estado de salud del backend se consulta en `https://proyecto-cafe-top-1.onrender.com/health`. Si Render suspende un servicio gratuito por inactividad, la primera petición puede tardar en despertar. Las limitaciones del plan, límites de almacenamiento y condiciones de cada proveedor pueden cambiar y deben confirmarse en sus paneles oficiales.

Si las imágenes se suben al disco local del servidor, pueden perderse cuando el servicio gratuito se reinicia o se vuelve a desplegar. Para guardar archivos de forma duradera se requiere un servicio de almacenamiento persistente correctamente configurado. No se deben subir archivos `.env`, contraseñas, bases locales ni exportaciones privadas al repositorio.

La URL pública permite presentar el frontend y la ruta `/health` proporciona un punto de comprobación del servidor. La existencia de esas direcciones no prueba por sí sola el envío correcto de un pedido, el acceso con todos los roles ni la disponibilidad permanente. Esos pasos se deben ensayar antes de grabar y mostrar en el vídeo.

23. Pruebas realizadas

La verificación del proyecto combina pruebas automatizadas, compilación y recorridos manuales. Las pruebas automatizadas disponibles usan `node:test` en los módulos de JavaScript. En la revisión del código se ejecutaron 18 pruebas de utilidades del frontend y 5 pruebas de imágenes del backend; todas terminaron correctamente. La compilación de producción del frontend también terminó correctamente.

Se realizaron además comprobaciones manuales en una vista previa local para la selección de un producto, la incorporación al carrito, la animación hacia el icono, la apertura del panel y el regreso al menú. Se observó que la imagen probada cargaba desde un archivo local. Estas pruebas describen el build local revisado; antes de grabar se debe comprobar el mismo comportamiento en la URL publicada.

Las pruebas de funciones aisladas no reemplazan la verificación de producción. Una compilación correcta no demuestra que Render tenga variables correctas, que MongoDB esté disponible, que CORS acepte el dominio publicado o que todos los roles tengan permisos adecuados.

Comprobación	Resultado observado	Alcance
Build de Vite	Correcto	Compilación local
Utilidades del frontend	18/18 correctas	Pruebas automatizadas
Imágenes backend	5/5 correctas	Pruebas automatizadas
Flujo carrito y menú	Correcto en vista previa	Prueba manual local
Flujos integrales en producción	No incluidos en las pruebas locales enumeradas	Requieren comprobación en la URL publicada

24. Seguridad y privacidad

El backend utiliza Helmet para varias cabeceras HTTP, un limitador de solicitudes para la API, configuración de CORS y validación de datos en determinadas operaciones. Las contraseñas se procesan con bcrypt y la API usa tokens JWT para reconocer sesiones. Estas prácticas reducen algunos riesgos, pero no convierten al proyecto en un sistema certificado ni eliminan la necesidad de revisar el código.

Los secretos deben vivir en variables privadas del servidor y nunca en Git o en el bundle de React. Las claves de servicios externos no deben copiarse a capturas, grabaciones ni memorias públicas. Las cuentas que se utilicen en la demo deben ser de prueba y no reutilizar contraseñas personales.

La aplicación maneja datos de contacto en reservas y perfiles. Solo se deben recoger los datos necesarios y limitar quién puede consultarlos. Las capturas del proyecto deben ocultar correos, teléfonos, direcciones y cualquier identificador real. Para uso comercial sería necesario revisar aviso de privacidad, retención y eliminación de datos, copias de seguridad y respuesta ante incidentes.

Una revisión posterior debe verificar con pruebas que cada endpoint privado rechaza peticiones sin sesión y roles no autorizados, que se validan entradas, que los errores no exponen información sensible y que las sesiones se gestionan de forma adecuada. No se documenta aquí una auditoría externa porque no se ha realizado.

25. Accesibilidad y adaptación a pantallas

El sitio utiliza clases responsivas para reorganizar contenido y tarjetas según el ancho de pantalla. El carrito ocupa el ancho disponible en móvil y se presenta como panel lateral en pantallas más amplias. Los controles importantes son botones HTML, y algunas imágenes tienen texto alternativo.

La adaptación visual no garantiza por sí sola accesibilidad completa. Para mejorarla conviene revisar el contraste de textos sobre los fondos oscuros, el tamaño de letra, el orden de tabulación, el uso con teclado y las etiquetas de los controles. Los formularios deben indicar qué información se necesita y mostrar los errores cerca del campo correspondiente.

Las animaciones deben respetar a las personas que han solicitado movimiento reducido en su sistema operativo. La experiencia principal debe continuar funcionando aunque la animación no se reproduzca. La fotografía tampoco debe ser el único modo de identificar un producto: el nombre, la categoría y el precio deben permanecer visibles.

La comprobación más útil es recorrer el menú y el carrito en un teléfono real o en la emulación del navegador, probar zoom, orientación, teclado y lector de pantalla si se dispone de uno, y corregir problemas observados. Incluye solo los resultados que realmente se hayan probado.

La memoria no atribuye una certificación de accesibilidad ni resultados de lector de pantalla. El vídeo debe mantener el texto legible y mostrar controles sin tapar la interfaz.

26. Limitaciones actuales

El proyecto es una aplicación académica en evolución. Aunque cuenta con un despliegue y varios flujos, no se debe confundir ese resultado con una solución lista para procesar operaciones comerciales a gran escala.

Entre los puntos que requieren atención están las pruebas integrales en producción con cada tipo de cuenta, la verificación de permisos del panel, la recuperación y restauración de copias de seguridad, y la confirmación de persistencia de imágenes. También conviene revisar los mensajes y los estados de error cuando hay problemas de red o la API tarda en responder.

El modo sin conexión conserva algunos recursos previamente cargados, pero no garantiza el envío ni la sincronización de pedidos cuando vuelve internet. La disponibilidad de recomendaciones automatizadas depende de configuración externa. Las reservas son solicitudes cuyo estado puede necesitar confirmación del personal. La presencia de campos para pagos o fidelidad no significa que todos esos procesos se hayan probado con transacciones reales.

Las fotografías de menú deben revisarse visualmente y mantenerse actualizadas. Las imágenes ilustrativas no sustituyen una ficha confiable de ingredientes. Para información sobre alergias, la persona debe consultar directamente al personal.

Esta lista no invalida el trabajo realizado; delimita lo que se puede afirmar y ayuda a priorizar mejoras.

27. Decisiones y aprendizajes

Una decisión importante fue separar el frontend del backend y desplegarlos como servicios distintos. Esto permitió publicar los archivos de React en una plataforma adecuada para sitios estáticos y mantener la API Node en un servicio que ejecuta procesos de servidor. La conexión con MongoDB se configura mediante variables privadas.

Otra decisión fue ofrecer personalización por tipo de producto. La lógica debe evitar preguntas irrelevantes y conservar las selecciones en cada línea. Durante las pruebas se hizo evidente que la experiencia no termina al pulsar un botón: es necesario ver qué se agregó, poder comprobar el precio y llegar fácilmente de nuevo al menú.

El trabajo también mostró la importancia de distinguir datos locales y datos públicos. Que una imagen exista en el proyecto no significa que la base de datos la esté usando; por eso se añadió lógica para corregir rutas antiguas y un respaldo cuando una imagen no carga. Una modificación local, además, no llega automáticamente al sitio publicado: debe compilarse y desplegarse.

El proceso deja tres aprendizajes concretos: seguir el recorrido completo del cliente en lugar de validar controles aislados; comprobar datos e imágenes en el catálogo real, no solo en archivos locales; y desplegar y volver a probar cualquier cambio antes de presentarlo como parte de la versión pública.

28. Conclusiones y mejoras futuras

Café Exquisito reúne una experiencia de consulta de menú con funciones para elegir productos y organizar información de pedidos y reservas. La aplicación tiene una estructura que separa la interfaz de la API, emplea modelos para los datos y presenta una identidad visual consistente con la idea de una cafetería.

El principal aprendizaje del proyecto es que una interfaz debe conectar correctamente cada paso. No basta con mostrar una tarjeta bonita: la persona necesita entender las opciones, el precio, la confirmación y el camino para continuar navegando. También es importante probar la versión real que se va a presentar y no basarse solamente en la vista local.

Como mejoras futuras se propone completar pruebas de permisos con cuentas separadas, mejorar la gestión de imágenes, revisar accesibilidad con herramientas específicas, documentar copias de seguridad y medir la comprensión del flujo con usuarios voluntarios. Si se habilitan pagos, deben validarse en un entorno de prueba antes de recibir dinero real.

La versión actual debe presentarse con honestidad: se pueden describir los módulos desarrollados y los resultados de las pruebas locales, y se deben separar de las funciones que dependen de configuración o de verificaciones aún pendientes. Esa claridad permite explicar no solo qué hace la aplicación, sino también cómo se comprobó.

Como conclusión, el proyecto reúne una carta digital con personalización y flujos de atención en una aplicación con frontend, API y base de datos separados. La principal prioridad antes de la defensa es ensayar el mismo recorrido en el sitio publicado, comprobar que el menú llega desde el backend y explicar con claridad qué partes se probaron localmente y cuáles se observaron en producción.

Las siguientes mejoras razonables son completar pruebas de permisos con cuentas de demostración, revisar accesibilidad con herramientas específicas, documentar copias de seguridad y evaluar los flujos con personas usuarias. Los pagos reales solo deberían añadirse después de comprobar la integración en un entorno de prueba.

29. Guion del vídeo de defensa

La guía de entrega fija un máximo de cinco minutos. El vídeo debe estar narrado por el estudiante y mostrar el problema, una demostración real, las tecnologías y que React obtiene datos del backend. El guion siguiente dura aproximadamente cuatro minutos y cincuenta segundos para dejar margen al inicio y al cierre.

Guion de 4 minutos y 50 segundos

0:00–0:30 — Presentación y problema. Preséntate, di el nombre del proyecto y explica que la propuesta busca reunir la carta, la personalización y la revisión del pedido en un solo sitio, en vez de dispersar esa información.

0:30–1:00 — Aportación. Explica brevemente qué aporta el sitio: consultar categorías, elegir opciones pertinentes al producto y conservarlas en el carrito. No afirmes que se midieron mejoras de ventas o tiempos.

1:00–1:30 — Tecnologías. Menciona React y Vite en la interfaz, Node.js y Express en la API, y MongoDB con Mongoose para los datos. Resume el recorrido de una solicitud: React pide el menú a la API y la API responde con datos almacenados en MongoDB.

1:30–3:05 — Demostración conectada. Abre el sitio HTTPS publicado y muestra cómo carga el menú. Explica que los productos visibles vienen de la API; no muestres solo imágenes o código local como prueba. Elige un producto, abre sus opciones, selecciona una personalización pertinente y añádelo al carrito. Enseña las opciones y el total.

3:05–3:45 — Un caso adicional. Muestra una reserva, promoción o evento que realmente esté disponible. Si tienes una cuenta de prueba autorizada, enseña una acción sencilla del panel; si no, usa este tiempo para explicar el modelo de datos o la respuesta de la API sin exponer claves ni información personal.

3:45–4:20 — Seguridad y pruebas. Di que las contraseñas se guardan con hash y que el servidor controla rutas protegidas. Menciona las pruebas locales ejecutadas y aclara que no equivalen a probar todos los flujos de producción. Indica una limitación real, por ejemplo que no se afirma haber probado pagos reales ni sincronización de pedidos sin conexión.

4:20–4:50 — Cierre. Resume el resultado, nombra una mejora futura y agradece. Termina antes de los cinco minutos.

Antes de grabar

- Ensaya desde la URL pública y verifica que el menú responde con datos de la API.
- Prueba el producto, sus opciones, el carrito y cualquier caso adicional que vayas a mostrar.
- Usa datos y cuentas de prueba; no grabes contraseñas, tokens, correos ni datos personales.
- Narra con tus propias palabras y confirma que puedes explicar las decisiones técnicas.
- Revisa que el enlace del vídeo abra sin pedir permisos y agrega ese enlace en el comentario de la entrega.

Si un flujo falla durante el ensayo, corrígelo o elige otro que funcione. No ocultes el error con una grabación que dé a entender que la función operó correctamente.

30. Fuentes y declaración de uso de IA

Fuentes internas consultadas

- `README.md`: descripción, funciones y límites del proyecto.
- `DESPLIEGUE_GRATIS.md`: arquitectura de publicación e instrucciones de despliegue.
- `backend/server.js`: rutas y middleware registrados por la API.
- `backend/models/Product.js`, `Order.js`, `User.js` y `Reservation.js`: estructura de datos principal.
- `frontend/src/App.jsx`: navegación y composición general de la interfaz.
- `frontend/src/pages/Home.jsx`: portada y catálogo.
- `frontend/src/components/ProductCard.jsx` y `CartDrawer.jsx`: selección de productos y carrito.
- `frontend/src/context/CartContext.jsx`: estado del pedido en la interfaz.
- Pruebas automatizadas en `backend/test` y `frontend/src/utils`.

Uso de herramientas de inteligencia artificial

Durante el desarrollo y la preparación de esta memoria se utilizó asistencia de IA generativa como apoyo para revisar y redactar textos, proponer y ajustar código, investigar errores y organizar pruebas. Las sugerencias no se consideran pruebas por sí mismas: los cambios se incorporaron al proyecto y se contrastaron con el código, la compilación, las pruebas disponibles o una comprobación manual cuando correspondía.

La IA no sustituye la autoría ni la defensa del proyecto. El estudiante debe conocer el funcionamiento de lo que presenta, poder explicar las decisiones, reconocer las limitaciones y realizar modificaciones propias. Esta declaración describe el uso de herramientas de apoyo; no afirma que la IA haya validado la aplicación completa.

Qué se entrega en la plataforma

La URL del repositorio y la de la aplicación se colocan en sus campos correspondientes. El PDF se adjunta como archivo. El enlace al vídeo y las credenciales de prueba se agregan en el comentario de la entrega, siguiendo las instrucciones de la plataforma y utilizando una cuenta creada para la evaluación. No se incluyen contraseñas en esta memoria.

Fin de la memoria.